

Proj Streaming Platform

TECH STACK

- Golang as backend because the zero-overhead concurrency, can scale to millions of messages with cheap hosting
- Using Websocket to continuous open pipe, avoids the heavy overhead of repeating HTTP headers.
- Vue 3 + Vite for blazing-fast compilation, atomic DOM patching, and minimal initial bundle size.

PROTOTYPE

1. Dynamic Room Creation

A user lands on the homepage and click a single button “create watch room”. The Go backend generates a unique, short cryptographic string (e.g., watchparty.com/room/xyz123) and assigns that user as the “Host”. When someone joins the room, just prompt them with a simple text box like “Enter your nickname to join the chat”

2. Video Sync Core (Play/Pause/Seek)

For prototype, no Crunchyroll but use raw and public direct .mp4 link or sample video file hosted on a free CDN. Vue’s job is to bind event listeners to the native HTML5 <video> tag. Go’s job is to maintain the single source of truth for that room’s state. Create a lightweight JSON payload from Vue to Go whenever the host interacts with the player. When Go receives this, it instantly broadcast this exact payload out to every other WebSocket connection registered under roomId: xyz123

```
{
  "action": "SYNC_EVENT",
  "payload": {
    "roomId": "xyz789",
    "playerState": "PAUSED",
    "currentTime": 142.35,
    "sentAt": 1718987482
  }
}
```

3. Live Chat

A simple input box alongside the video player. When a user hits enter, the message is piped through the WebSocket. Do not save these messages to a

database yet, the Go server should just act as reflector, it receives the text packet and instantly spits it back out to everyone else in the room. If a user refreshes their browser, losing the chat history is completely fine for a prototype.

MVP DATA ARCHITECTURE (In Go Memory)

To keep this ultra-fast and avoid database configuration headaches on day one, you can store the active room states directly in Go's memory using a standard "map" protected by a sync.Mutex (to prevent data corruption from concurrent threads)

The data blueprint in Go will look something like this:

```
type User struct {
    ID      string
    Username string
    Conn    *websocket.Conn // The live network pipe
}

type WatchRoom struct {
    RoomID      string
    HostID      string
    CurrentVideo string // URL of the video file
    CurrentTime float64 // Latest timestamp in seconds
    IsPlaying   bool   // Play/Pause status
    Clients     map[string]*User // Everyone currently inside the room
}
```